

Árboles

Estructura de Datos



Programa 6

—

Alumno: Quijas Contreras Victor Manuel

Código: 220429771

Horario: Lun, Mie y Vie de 3:00 a 5:00 P.M.

Profesor: Noe Ortega Sánchez

Fecha: 03/11/2025

Árbol de Búsqueda Binaria

Un **árbol de búsqueda binaria** es un tipo de estructura de datos no lineal la cual contiene a todos sus elementos ordenados por una característica en común, este ordenamiento se lleva a cabo en el momento en el que se insertan los datos en el árbol. Sus datos se organizan de manera jerárquica y por niveles. **Jerárquica** porque un nodo puede descender y preceder de otro, y por **niveles**, porque se puede medir la altura de cada uno de los nodos de un árbol con respecto a su raíz.

Cada nodo cuenta con cero, uno o dos hijos, además de contar con un padre que lo precede a excepción del nodo raíz. Los nodos pueden ser catalogados como **nodo hoja**: nodo que no cuenta con hijos, **nodo rama**: nodo que tiene uno o más hijos y el **nodo raíz**: que no tienen predecesores.

Implementación de un ABB en C++

La implementación de un ABB es un poco más sencilla en algunos aspectos a comparación de las listas, pilas y colas. Donde más radica su complejidad es a la hora de comprender la recursividad y al eliminar uno de los nodos debido a la cantidad de casos que se pueden llegar a presentar.

Clase Nodo:

La clase nodo se define de la siguiente manera:

- **Info**: Tipo de dato.
- **L**: Apuntador al hijo izquierdo.
- **R**: Apuntador al hijo derecho.

En mi caso, el tipo de dato es una clase llamada "Dato" y puede guardar un entero, un flotante, un carácter y una cadena. Y lo que se va a utilizar para ordenar el árbol va a ser por el entero. Los apuntadores l y r se llaman así por su inicial en inglés "left" y "right".

```
1      #pragma once
2
3      #include "Dato.h"
4
5      class Nodo
6      {
7      public:
8          Dato* info;
9          Nodo* l;
10         Nodo* r;
11
12         Nodo();
13         Nodo(Dato* info);
14         Nodo(Dato* info, Nodo* l, Nodo* r);
15         ~Nodo();
16     };
17
```

Sus constructores contienen la siguiente definición:

Constructor base: Construye al nodo con todos sus valores inicializados en nullptr. Cuando se necesita preparar un nuevo nodo, pero se desconoce la información y la posición en la que se va a insertar.

```
1      #include "Nodo.h"
2
3      /*Constructores*/
4      ~ Nodo::Nodo() {
5          this->info = nullptr;
6          this->l = nullptr;
7          this->r = nullptr;
8      }
9
```

```
10     ~ Nodo::Nodo(Dato* info) {
11         this->info = info;
12         this->l = nullptr;
13         this->r = nullptr;
14     }
```

Constructor con parámetro de valor:

Construye el nodo con la dirección de un objeto Dato como se puede ver en la línea 11. Mientras que mantiene sus apuntadores a otros nodos en nullptr.

Constructor con todos los parámetros: Crea al nodo con todo inicializado, asigna un puntero a su parte de info, a su parte l y a su parte r como se puede ver de la línea 17 a la 20.

```
16     ~ Nodo::Nodo(Dato* info, Nodo* l, Nodo* r) {
17         this->info = info;
18         this->l = l;
19         this->r = r;
20     }
```

Clase Árbol:

```
1      #pragma once
2      #include "Nodo.h"
3
4      ~ class Arbol
5      {
6      private:
7          ~ Nodo* raiz;
8          ~ int tamaño;
```

La clase árbol depende de la clase nodo para operar.

Sus principales atributos son:

- **Raíz:** Variable tipo puntero donde se guardará el principal acceso a nuestro arreglo de datos.
- **Tamaño:** Variable de tipo entero que guardará el tamaño de los nodos que existan dentro del árbol.

Con estas dos variables obtenemos el acceso a nuestro árbol, además de facilitarnos la creación de una función que devuelva el tamaño sin tener que recorrer el árbol.

Cabe aclarar que el árbol está ordenado por la parte entera de los Datos. Más adelante se dejará explícito al explicar la clase Dato. Las operaciones del árbol están divididas en tipos dependiendo su función: funciones públicas, función de insertar, funciones de búsqueda, funciones de eliminación.

Comentado [VC1]: Explicar este pedo

Las operaciones del árbol son las siguientes:

Función de insertar

La función *Insertar* lleva como **parámetros referencia**, que es un puntero de tipo Nodo y **info** que es otro puntero de tipo Dato y devuelve un puntero de tipo Nodo.

```
17  /*Operaciones*/  
18  Nodo* Insertar(Nodo* referencia, Dato* info);
```

La función utiliza recursividad. Al llamarla, se le pasará como parámetro la raíz y un objeto dato ya construido. Entonces:

1. Va a **verificar** que la referencia exista, si no existe ninguna referencia en ese espacio entonces la función:
 - a. **Crea** un Nodo auxiliar.
 - b. **Incrementa** en 1 la variable tamaño.
 - c. **Retorna** el auxiliar.
2. Si existe un nodo en ese espacio entonces va a comparar info en su parte entera, con la parte entera de la referencia. Si es menor a la de la referencia entonces la función:
 - a. **Asignará a la parte izquierda** del nodo referencia la función *Insertar* pero ahora, tomará como puntero la parte izquierda de la referencia e igual pasa el mismo puntero dato.
3. En cambio, si es mayor la función:
 - a. **Asignará a la parte derecha** del nodo referencia la función *Insertar* pero ahora, tomará como puntero la parte derecha de la referencia e igual pasa el mismo puntero dato.
4. Una vez alguna función de la pila haya caído en la primer condición, al final de la función va a **retornar la misma referencia**. Esto hace que la función:
 - a. **Enlace** el nuevo nodo insertado como hijo del nodo anterior que lo llamó.
 - b. Cada retorno va a **devolver el subárbol actualizado** hasta terminar la pila de llamadas que usó la función.

```

82 //Insertar
83 void Arbol::Insertar(Nodo* referencia, Dato* info) {
84     if (!referencia) {
85         Nodo* aux = new Nodo(info);
86         tamano++;
87         return aux;
88     }
89
90     if (info->GetEntero() < referencia->info->GetEntero())
91         referencia->l = Insertar(referencia->l, info);
92     else
93         referencia->r = Insertar(referencia->r, info);
94
95     return referencia;
96 }

```

Función insertar

Funciones de Mostrar

En las funciones de mostrar tenemos los métodos:

```

20 void MostrarInOrder(Nodo* referencia);
21 void MostrarPreOrder(Nodo* referencia);
22 void MostrarPosOrder(Nodo* referencia);
23 void MostrarNodo(Nodo* referencia);
24 void CopiarDatos(Nodo* referencia, Nodo* heredero);

```

MostrarInOrder: Lleva como **parámetro referencia**, un puntero nodo. Su comportamiento es el siguiente:

1. **Verifica** que exista la referencia, si no existe un nodo:
 - a. **Retorna**.
2. **Recorre** el subárbol izquierdo.
3. Una vez cumplió con la primera condición, **imprime** lo que exista en el nodo.
4. **Recorre** el subárbol derecho.

De esta forma se pasa por todos los nodos en orden utilizando recursividad. Esto funciona debido que una vez que llega al límite izquierdo, es decir el dato de mínimo valor, imprime este dato, después recorre toda la parte derecha de este dato, es decir, sus sucesores. Si no tiene entonces se regresará a su predecesor, que imprimirá la contenga. Ahora estarán impresos dos números en orden, este proceso se va a repetir hasta que se llegué al límite derecho, esto muestra todo el contenido del árbol de manera ascendente.

```

98 //Mostrar
99 void Arbol::MostrarInOrder(Nodo* referencia) {
100     if (!referencia) {
101         return;
102     }
103     MostrarInOrder(referencia->l);
104     cout << referencia->info->GetEntero() << "->";
105     MostrarInOrder(referencia->r);
106 }

```

MostrarPreOrder: Lleva como **parámetro referencia**, un puntero nodo. Su comportamiento es el siguiente:

1. **Verifica** que exista la referencia, si no existe un nodo:
 - a. **Retorna**.
2. **Imprime** lo que exista en el nodo.
3. **Recorre** el subárbol izquierdo.
4. **Recorre** el subárbol derecho.

Esto lo que ocasiona es que primero muestre los padres y después a su descendencia, permitiendo ver cómo es que está estructurado el árbol.

```
108 void Arbol::MostrarPreOrder(Nodo* referencia) {
109     if (!referencia) {
110         return;
111     }
112     cout << referencia->info->GetEntero() << "->";
113     MostrarPreOrder(referencia->l);
114     MostrarPreOrder(referencia->r);
115 }
```

MostrarPosOrder: Lleva como **parámetro referencia**, un puntero nodo. Su comportamiento es el siguiente:

1. **Verifica** que exista la referencia, si no existe un nodo:
 - a. **Retorna**.
2. **Recorre** el subárbol izquierdo.
3. **Recorre** el subárbol derecho.
4. **Imprime** lo que exista en el nodo.

Esto permite primero visitar a los hijos de cada nodo antes que al padre, útil para una función que se verá más adelante.

```
117 void Arbol::MostrarPosOrder(Nodo* referencia) {
118     if (!referencia) {
119         return;
120     }
121     MostrarPosOrder(referencia->l);
122     MostrarPosOrder(referencia->r);
123     cout << referencia->info->GetEntero() << "->";
124 }
```

MostrarNodo: Lleva como **parámetro** un puntero nodo **referencia**, y su función es la siguiente:

1. **Valida** si la referencia existe, si no existe:
 - a. **Imprime** una alerta, diciendo que no existe.
 - b. **Retorna**.
2. Si existe, entonces **imprime** un marco que separa el texto anterior para darle forma.
3. **Imprime** la parte **entera** del dato.
4. **Imprime** la parte **flotante** del dato.
5. **Imprime** la parte del **carácter** del dato.
6. **Imprime** la parte tipo **cadena** del dato.
7. **Imprime** el fin del margen.
8. **Retorna**.

```
126 void Arbol::MostrarNodo(Nodo* referencia) {
127     if (!referencia) {
128         cout << "MostrarNodo: Referencia apuntando a nulo." << endl;
129         return;
130     }
131
132     cout << endl << "-----" << endl;
133     cout << "Entero: " << referencia->info->GetEntero() << endl;
134     cout << "Flotante: " << referencia->info->GetFlotante() << endl;
135     cout << "Caracter: " << referencia->info->GetCaracter() << endl;
136     cout << "Cadena: " << referencia->info->GetCadena() << endl;
137     cout << "-----" << endl;
138     return;
139 }
```

CopiarDatos: Lleva como **parámetros** dos punteros a nodo, el puntero **referencia** es el nodo que se le va a actualizar y puntero **heredero**, lleva ese nombre por el lugar donde se utiliza esta función. La función se comporta de la siguiente manera:

Utiliza el setter de la referencia para copiar la información que devuelve el getter del heredero.

1. **Copia** el entero del heredero y lo **pega** en la referencia.
2. **Copia** el flotante del heredero y lo **pega** en la referencia.
3. **Copia** el carácter del heredero y lo **pega** en la referencia.
4. **Copia** la cadena del heredero y lo **pega** en la referencia.

```
141 void Arbol::CopiarDatos(Nodo* referencia, Nodo* heredero) {
142     referencia->info->SetEntero(heredero->info->GetEntero());
143     referencia->info->SetFlotante(heredero->info->GetFlotante());
144     referencia->info->SetCaracter(heredero->info->GetCaracter());
145     referencia->info->SetCadena(heredero->info->GetCadena());
146     return;
147 }
```

Funciones de búsqueda

Dentro de las funciones de búsqueda tenemos:

```
26     bool Vacio();
27     int Tamano();
28     Nodo* Buscar(Nodo* referencia, int entero);
29     Nodo* Heredero(Nodo* referencia);
```

Vacio: Retorna un valor booleano que depende de:

1. Si la raíz no contiene nada, es decir, el árbol está vacío, entonces **devuelve verdadero**.
2. Si no, **devuelve falso**.

```
152     bool Arbol::Vacio() { return raiz == nullptr; }
```

Tamano: Retorna el valor un entero.

1. **Retorna** el tamaño.

```
150     int Arbol::Tamano() { return tamano; }
```

Buscar: Retorna un puntero nodo, y recibe como **parámetros** un puntero nodo **referencia**, y el **entero**, el cuál es el que se desea encontrar. Para esto la función:

1. **Verifica** si la referencia existe, si no existe:
 - a. **Imprime** una alerta advirtiendo que no existe.
 - b. **Retorna** nullptr.
2. **Verifica** si el entero es igual a la parte entero de la referencia, si ya se encontró el entero, entonces:
 - a. **Retorna** referencia.
3. Si no, **verifica** si es menor que la referencia, si es menor:
 - a. **Retorna** lo que devuelva la función buscar y pasa de parámetro la referencia en su parte izquierda.
4. Si no, **verifica** si es mayor que la referencia, si es mayor:
 - a. **Retorna** lo que devuelva la función buscar y pasa de parámetro la referencia en su parte derecha.

Funciona de manera recursiva y con búsqueda binaria, para reducir significativamente el número de operaciones que realiza para encontrar el entero.


```

154  ✓ Nodo* Arbol::Buscar(Nodo* referencia, int entero) {
155  ✓      if (!referencia) {
156          cout << "Buscar: No existe el elemento en la lista!" << endl;
157          return nullptr;
158      }
159
160  ✓      if (referencia->info->GetEntero() == entero) {
161          return referencia;
162      }
163  ✓      if (referencia->info->GetEntero() > entero) {
164          return Buscar(referencia->l, entero);
165      }
166  ✓      else {
167          return Buscar(referencia->r, entero);
168      }
169  }
170

```

Heredero: Esta función es muy importante para la operación de eliminar, su **parámetro es** un nodo **referencia**. Y funciona de la forma:

1. **Verifica** si la referencia existe, si no existe entonces significa el elemento anterior enviado era un nodo hoja. Entonces:
 - a. **Imprime** una alerta.
 - b. **Retorna** nullptr.
2. Si existe una referencia comienza un ciclo de recursividad. **Verifica** si existe la referencia en su parte derecha. Si existe:
 - a. **Retorna** el puntero que devuelva la función heredero.
3. Si no, es decir, ya no existe un elemento más grande;
 - a. **Retorna** el mismo apuntador de referencia.

Esto hará que toda la pila de llamadas devuelva el mismo nodo, el cuál será considerado el heredero al trono.

```

171  ✓ Nodo* Arbol::Heredero(Nodo* referencia) {
172  ✓      if (!referencia) {
173          cout << "Heredero: Referencia apuntando a nulo!" << endl;
174          return nullptr;
175      }
176
177      if (referencia->r)
178          return Heredero(referencia->r);
179      else
180          return referencia;
181  }

```

Funciones de eliminar

La parte más compleja de explicar es la función de eliminar un nodo en específico. Pues, se presentan una serie de diversos casos que complican la acción.

Eliminar: Para **poder eliminar necesitamos observar** primero, cuáles son las formas en las que se puede presentar un nodo. Como ya lo mencionamos en la introducción, un nodo puede ser **hoja, rama o raíz**. Dependiendo de que sea la referencia, es el método que vamos a utilizar. Pero primero, desglosemos paso a paso, lo que este método realiza.

Como parámetros recibe dos variables: **referencia**, un puntero a nodo y **entero**, el valor que se busca eliminar. Entonces nuestro método:

1. **Verifica** si la referencia existe. Si no existe:
 - a. **Retorna** nullptr.
2. En cambio, inicia el proceso de comparaciones. **Pregunta** si el entero es menor a la parte entera de la referencia, si es menor entonces:
 - a. **Asigna** a la parte de la izquierda de la referencia **el subárbol** que se va a actualizar con la función de eliminar.
3. Si no, **pregunta** si el entero es mayor, si es mayor entonces:
 - a. **Asigna** a la parte de la derecha de la referencia **el subárbol** que se va a actualizar con la función de eliminar.
4. Si no, significa que el valor es lo que se desea eliminar y entramos a los casos de eliminar.
 - a. **Verifica si** la referencia **es un nodo hoja**, es decir que no tenga hijos. Si es un nodo hoja entonces:
 - i. El tamaño **decrece**.
 - ii. Se **elimina** la referencia del árbol.
 - iii. **Retorna** nullptr.
 - b. Si no es hoja, entonces **verifica si es un nodo rama** con un solo hijo. Si es así, entonces:
 - i. **Pregunta** si existe un hijo izquierdo. Si existe:
 1. Tamaño **decrece**.
 2. Se **crea un puntero temporal** con el valor del hijo izquierdo de la referencia.
 3. Se **elimina** la referencia.
 4. **Retorna** el temporal.
 - ii. Si no, pregunta si existe un hijo derecho. Si existe:
 1. Tamaño **decrece**.
 2. Se **crea un puntero temporal** con el valor del hijo derecho de la referencia.
 3. Se **elimina** la referencia.
 4. **Retorna** el temporal.

- c. Si no, entonces **verifica si** lo que se tiene de referencia **es un nodo rama con ambos hijos**, para este caso es donde se utiliza la función especial heredero. Si efectivamente es así, entonces:
- Crea un puntero llamado heredero**, al cual se le va a asignar el valor del puntero a nodo que retorne la función con el mismo nombre. Para que la función haga su trabajo correctamente es **necesario pasarle como argumento la parte izquierda de la referencia**.
 - Llama a la función que copia los datos**. Se pasa de argumento a la heredero, en ese respectivo orden. Y realiza su operación.
 - Ahora la función va a **eliminar al heredero** que quedó como copia en el árbol, para esto a la parte izquierda de la referencia, que es donde se encuentra el heredero, se le va a asignar el subárbol actualizado que devuelva la función eliminar. Esto va a iniciar otra búsqueda recursiva del heredero para eliminarlo **físicamente del árbol**.
5. Al final, para que cada llamada de la función devuelva el subárbol actualizado es necesario **retornar la propia referencia que la llamó**. Esto termina con la pila de llamadas que se realizó.

```

185  ✓  Nodo* Arbol::Eliminar(Nodo* referencia, int entero) {
186  ✓      if (!referencia) {
187  ✓          return nullptr;
188  ✓      }
189  ✓
190  ✓      if (entero < referencia->info->GetEntero()) {
191  ✓          referencia->l = Eliminar(referencia->l, entero);
192  ✓      }
193  ✓      else if (entero > referencia->info->GetEntero()){
194  ✓          referencia->r = Eliminar(referencia->r, entero);
195  ✓      }
196  ✓      else {
197  ✓          if (!referencia->l && !referencia->r) { //Nodo hoja
198  ✓              tamano--;
199  ✓              delete referencia;
200  ✓              return nullptr;
201  ✓          }
202  ✓          else if (!referencia->l || !referencia->r){ //Nodo rama
203  ✓              if (referencia->l) {
204  ✓                  tamano--;
205  ✓                  Nodo* temp = referencia->l;
206  ✓                  delete referencia;
207  ✓                  return temp;
208  ✓              }
209  ✓              else if (referencia->r) {
210  ✓                  tamano--;
211  ✓                  Nodo* temp = referencia->r;
212  ✓                  delete referencia;
213  ✓                  return temp;
214  ✓              }
215  ✓          }
216  ✓          else if (referencia->l && referencia->r) { //Nodo rama (ambos hijos)
217  ✓              Nodo* heredero = Heredero(referencia->l);
218  ✓              CopiarDatos(referencia, heredero);
219  ✓              referencia->l = Eliminar(heredero, heredero->info->GetEntero());
220  ✓          }
221  ✓      }
222  ✓      return referencia;
223  ✓  }

```

EliminarArbol: Eliminar árbol lleva como **parámetro** una **referencia** a un nodo. Es de las funciones más sencillas pues utiliza una estructura que anteriormente se había mencionado, es decir utilizamos la misma estructura de atravesar el árbol en PostOrder, donde primero visitamos a todos los hijos de los nodos y al último los padres. Para eliminar hijo por hijo sin tener fugas de datos. Su método se ve de la siguiente manera:

1. Si la **referencia no existe**, entonces:
 - a. **Retorna**.
2. Si **existe**, **recorre** todo el subárbol izquierdo.
3. Después, **recorre** todo el subárbol derecho.
4. Una vez que encuentre un nodo hoja, lo va a **eliminar** y **retornar**.

Esto se repite hasta que todos los nodos del árbol hayan sido eliminados.

```
225 void Arbol::EliminarArbol(Nodo* referencia) {
226     if (!referencia) {
227         return;
228     }
229
230     EliminarArbol(referencia->l);
231     EliminarArbol(referencia->r);
232     delete referencia;
233 }
```

Funciones públicas

Para utilizar nuestras operaciones principales del árbol es necesario crear funciones públicas que puedan acceder a ellas mediante una interfaz, debido a que nuestra variable raíz (utilizada en casi todas las funciones) está encapsulada. Las cuatro funciones públicas son las siguientes:

PubInsertar: Como **parámetro** lleva **info**; un puntero de tipo dato, el cual es el que lleva toda la información de nuestro nodo. Su método es el siguiente.

```
16 void Arbol::PubInsertar(Dato* info) {
17     raiz = Insertar(raiz, info);
18     return;
19 }
```

1. **Asigna** a la raíz el árbol actualizado que va a devolver la función Insertar, y como parámetros manda la propia raíz y el puntero dato.
2. **Retorna**.

PubMostrar: Como parámetro lleva eleccion; un entero que representa la elección de la función que se desea utilizar.

```
21 void Arbol::PubMostrar(int eleccion) {
22     switch (eleccion) {
23     case 1:
24         MostrarInOrder(raiz); //1.MostrarInOrder
25         cout << "x" << endl;
26         break;
27     case 2:
28         MostrarPreOrder(raiz); //2.MostrarPreOrder
29         cout << "x" << endl;
30         break;
31     case 3:
32         MostrarPosOrder(raiz); //3.MostrarPosOrder
33         cout << "x" << endl;
34         break;
35     default:
36         cout << "Error al escoger la eleccion en Mostrar!" << endl;
37         break;
38     }
39     return;
40 }
```

Entra al menú donde, si **elección es:**

1. **Muestra** el árbol InOrder.
2. **Muestra** el árbol por medio del PreOrder.
3. **Muestra** el árbol por medio del PostOrder.

Si no, **imprime** una alerta que advierte que el valor de la elección no está permitido.

PubBuscar: Como parámetro lleva eleccion; tipo entero y un entero, que es el valor que se busca en caso de seleccionar el método de buscar.

Entra al menú donde, si **elección es:**

1. **Muestra** de manera segura con el método MostrarNodo lo que devuelva la función Buscar; con el parámetro entero.
2. **Imprime** el tamaño.
3. **Imprime** si el árbol está vacío o contiene elementos.

Si no, **imprime** una alerta que advierte que el valor de la elección no está permitido.

```
42 void Arbol::PubBuscar(int eleccion, int entero) {
43     switch (eleccion) {
44     case 1:
45         MostrarNodo(Buscar(raiz, entero)); //1.Buscar
46         break;
47     case 2: {
48         int x = Tamano(); //2. Tamaño
49         cout << "La cantidad de nodos en el arbol es de: " << x;
50         break;
51     }
52     case 3:
53         if (Vacio()) //3.Vacio
54             cout << "El arbol se encuentra vacio!" << endl;
55         else
56             cout << "El arbol cuenta con elementos." << endl;
57         break;
58     default:
59         cout << "Error al escoger la eleccion en Mostrar!" << endl;
60         break;
61     }
62 }
```

PubEliminar: Como parámetro lleva nuevamente eleccion y un entero. Esto debido a que la función Eliminar se comporta como una búsqueda.

Entra al menú donde, si **elección es:**

1. **Asigna** a la raíz el subárbol actualizado que devuelva la función de Eliminar, que lleva como parámetro la propia raíz y el entero que se desea eliminar.
2. **Elimina el árbol.**

Si no, **imprime** una alerta que advierte que el valor de la elección no está permitido.

```
64 void Arbol::PubEliminar(int eleccion, int entero) {
65     switch (eleccion) {
66     case 1:
67         raiz = Eliminar(raiz, entero); //1. Eliminar nodo en específico
68         break;
69     case 2:
70         EliminarArbol(raiz); //2. Eliminar árbol
71         break;
72     default:
73         cout << "Error al escoger la eleccion en Mostrar!" << endl;
74         break;
75     }
76 }
```

Estas son todas las operaciones que realiza mi implementación de un ABB en C++, como tal no utilicé un prompt en específico para generar el código, pero sí para resolver algunas dudas respecto a conceptos. Al final dejaré las referencias bibliográficas en las cuales me basé principalmente.

Clase Dato

Para cerrar, explicaré como está definida mi clase dato que se utilizó durante todas las operaciones del ABB.

Sus atributos son los siguientes:

Guarda cuatro tipos distintos de datos.

- Entero.
- Flotante.
- Carácter.
- Cadena.

Datos que se encuentran encapsulados. Es por eso por lo que se utiliza un método para obtenerlos y para asignarles valores.

```
3     #include <iostream>
4     #include <string>
5     using namespace std;
6
7     class Dato
8     {
9     private:
10         int entero;
11         float flotante;
12         char caracter;
13         string cadena;
```

Sus setters funcionan de la siguiente manera:

```
/*Setters*/  
void Dato::SetEntero(int entero) {  
    this->entero = entero;  
}  
void Dato::SetFlotante(float flotante) {  
    this->flotante = flotante;  
}  
void Dato::SetCaracter(char caracter) {  
    this->caracter = caracter;  
}  
void Dato::SetCadena(string cadena) {  
    this->cadena = cadena;  
}
```

Cada uno **obtiene** un su tipo de valor en específico **como parámetro**, y lo **guarda** en su parte de dato correspondiente.

Los getters son igual de sencillos:

```
/*Getters*/  
int Dato::GetEntero() { return entero; }  
float Dato::GetFlotante() { return flotante; }  
char Dato::GetCaracter() { return caracter; }  
string Dato::GetCadena() { return cadena; }
```

Únicamente **retornan el valor** que existe en esa parte específica.

La forma de sus constructores es la siguiente, y contiene tres.

El constructor **base**, **inicializa** sus valores **a cero**.

El constructor con un solo **entero** inicializa solo la parte entera. Útil pues, **nuestro árbol se ordena por su parte entera**.

El constructor **con todas sus partes inicializadas**, que es aquí donde entran los setters.

```
/*Constructores*/  
Dato::Dato(int entero, float flotante, char caracter, string cadena){  
    SetEntero(entero);  
    SetFlotante(flotante);  
    SetCaracter(caracter);  
    SetCadena(cadena);  
}  
  
Dato::Dato(int entero) {  
    SetEntero(entero);  
    this->flotante = 0.0;  
    this->caracter = ' ';  
    this->cadena = " ";  
}  
  
Dato::Dato() {  
    this->entero = 0;  
    this->flotante = 0.0;  
    this->caracter = ' ';  
    this->cadena = " ";  
}  
  
Dato::~Dato() {}
```

Conclusiones

El árbol de búsqueda binaria es una TDA bastante interesante pues se aleja de la linealidad que las listas, pilas y colas ofrecen, y te obliga a entender su funcionalidad por medio de la recursión. Su utilidad es remarcable pues, mejora mucho el procesamiento de búsqueda y gestión de los datos. Aunque puede ser un poco más compleja de entender su implementación es rápida y sencilla. Considero que este tipo de dato abstracto es perfecto para introducir a los contenedores no lineales.

Bibliografía

- W3Schools.com. (s. f.). https://www.w3schools.com/dsa/dsa_data_binarytrees.php
- *09 TDA Árbol binario*. (2019, 13 mayo). [Diapositivas; Electrónico PDF]. Sitio Web Docente Prof. Edgardo Adrián Franco Martínez en el IPN. <https://docencia.cafranco.com/materiales/estructurasdedatos/09/Tema09.pdf>
- Jenny's Lectures CS IT. (2019, 20 enero). *5.5 Binary Tree Traversals (Inorder, Preorder and Postorder) | Data structures and algorithms* [Video]. YouTube. <https://www.youtube.com/watch?v=-b2lciNd2L4>